

LAPORAN PRAKTIKUM PROGRAMAN WEB DAN SISTEM DATABASE



DOSEN PENGAMPU:

Abdul malik, S.kom. M.Cs

DISUSUN OLEH:

1. Muh. Sahrul Ibrahim
2. Alda Miranda
3. Sintha Lestari sasa
4. Rahel Rianti Suri

PROGRAM STUDI INFORMATIKA FAKULTAS ILMU KOMPUTER

UNIVERSITAS MEGABUANA PALOPO

Tahun ajara 2026/2027

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era transformasi digital saat ini, efisiensi dan keakuratan operasional pengelolaan data menjadi kunci utama dalam menjaga produktivitas suatu instansi atau perusahaan. Salah satu aspek mendasar yang memerlukan manajemen yang terstruktur adalah pengelolaan data pegawai dan rekapitulasi penggajian bulanan. Banyak institusi skala menengah atau organisasi internal yang hingga kini masih mempunyai pencatatan manual atau penggunaan aplikasi *spreadsheet* secara terpisah. Pendekatan ini memiliki kerentanan yang tinggi terhadap timbulnya redundansi data (duplikasi), inkonsistensi informasi konvensional ketika terjadi perubahan struktur organisasi, serta lambatnya proses pembuatan laporan berkala yang akurat.

Untuk mengatasi permasalahan tersebut, integrasi antara desain database relasional yang optimal dan implementasi aplikasi berbasis web menjadi solusi yang mutlak diperlukan. Melalui tugas proyek terintegrasi ini, dibangun sebuah **Sistem Informasi Pengelolaan Data Pegawai** yang menerapkan arsitektur database terstruktur hingga memenuhi standar Bentuk Normal Ketiga (3NF).

Proses pembangunan dan pengujian database dilakukan secara langsung menggunakan baris perintah (CLI) pada **Command Prompt (CMD)** melalui server lokal MySQL XAMPP. Langkah ini mencakup penulisan instruksi DDL (*Data Definition Language*) untuk mendefinisikan skema lima tabel saling berhubungan, DML (*Data Manipulation Language*) untuk manipulasi data, hingga implementasi objek database tingkat lanjut seperti *View* dan *Index* . Lebih jauh lagi, laporan ini juga menekankan aspek keamanan data menggunakan DCL (*Data Control Language*) berupa enkripsi hak akses pimpinan (GRANT/ REVOKE) serta aspek integritas transaksi data berlapis menggunakan mekanisme COMMIT dan ROLLBACK. Dengan sistem integrasi ini, seluruh aktivitas CRUD (*Create, Read, Update, Delete*) serta rekapitulasi anggaran total gaji dapat dieksekusi secara instan, aman, dan bebas dari galat (*error*).

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, maka rumusan masalah dalam proyek uas terintegrasi ini adalah sebagai berikut:

1. Bagaimana merancang skema database relasional yang terdiri dari minimal 5 tabel yang saling terhubung dan terbebas dari redundansi data menggunakan implementasi bentuk normal (3NF)?
2. Bagaimana mengimplementasikan perintah DDL, DML, *View* , dan *Index* secara manual melalui Command Prompt (CMD) untuk mengoptimalkan kinerja pencarian dan pemanggilan data pada aplikasi?

3. Bagaimana menerapkan aspek kendali keamanan (*Data Control Language*) menggunakan mekanisme GRANT/ REVOKE untuk membatasi hak akses pengguna pimpinan pada database?
4. Bagaimana memastikan integritas pencatatan data ganda yang saling berkaitan (pegawai dan gaji awal) terlindungi dari risiko kegagalan sistem melalui fitur transaksi database (COMMIT dan ROLLBACK)?
5. Bagaimana menghubungkan database MySQL yang telah dibuat dengan antarmuka web berbasis PHP murni agar fungsi CRUD dan laporan rekapitulasi pengeluaran gaji dapat berjalan secara dinamis?

1.3 Tujuan Proyek

Adapun tujuan yang ingin dicapai oleh kelompok kami dalam pengerjaan proyek uas ini adalah:

1. Menghasilkan rancangan cetak biru database (*Entity-Relationship Diagram*) dan struktur tabel riil yang memenuhi standar normalisasi database akademis.
2. Membangun dan menguji seluruh proses *script* SQL secara baris-per-baris melalui terminal Command Prompt (CMD) hingga berstatus *Query OK* .
3. Mengamankan data sensitif perusahaan (seperti nominal gaji pokok dan tunjangan) dengan mengisolasi hak akses menggunakan pengguna khusus `pimpinan_apps` .
4. Menerapkan proteksi kegagalan input data melalui skenario transaksi database, sehingga jika terjadi *error* di tengah jalan, database dapat dikembalikan bersih menggunakan perintah ROLLBACK.
5. Membuka aplikasi pengelolaan data kepegawaian berbasis web dengan tata letak folder yang rapi (termasuk folder `assets/` khusus CSS dan JS) untuk menampilkan hasil query database dalam bentuk visual yang interaktif.

1.4 Manfaat Proyek

Pengerjaan sistem informasi proyek dan database kepegawaian ini memberikan manfaat yang signifikan bagi beberapa pihak, antara lain:

- **Bagi Mahasiswa (Kelompok Pengembang):** Meningkatkan pemahaman praktis mengenai sinkronisasi mata kuliah Sistem Database dan Pemrograman Web, serta melatih kemampuan penyusunan kode program yang rapi, modular, dan terstruktur di GitHub.
- **Bagi Institusi / Pengguna Sistem:** Mempermudah bagian administrasi (HRD dan *Finance*) dalam melakukan pendataan pegawai baru, mengubah riwayat jabatan, menghapus data secara aman, serta melihat total anggaran pengeluaran gaji secara otomatis tanpa risiko salah hitung.

- **Bagi Aspek Akademis:** Menjadi contoh studi kasus penerapan digitalisasi sistem internal yang memanfaatkan fitur-fitur mutakhir database relasional (seperti *View* , *Index* , DCL, dan Transaksi) secara komprehensif.

1.5 Batasan Proyek

Mengingat keterbatasan waktu dan ruang lingkup penilaian UAS, proyek ini dibatasi oleh beberapa ruang lingkup berikut:

1. Database yang dirancang khusus berskala lokal (*localhost*) dengan menggunakan komponen bawaan MySQL dari XAMPP Control Panel.
2. Struktur database dibatasi pada 5 tabel utama yang meliputi tabel pegawai,,,, dan tabel .jabatandepartemengajipengguna
3. Akses eksekusi perintah keamanan (DCL) dan blok transaksi (COMMIT/ ROLLBACK) disimulasikan secara manual melalui terminal CLI Command Prompt, bukan melalui tombol di antarmuka web.
4. Aplikasi web dikembangkan menggunakan bahasa pemrograman PHP murni (*Native PHP*) tanpa menggunakan framework eksternal (seperti Laravel atau CodeIgniter) untuk menonjolkan pemahaman logika dasar pemrograman.
5. Fitur laporan pada file laporan.phphanya mencakup rekapitulasi total pengeluaran kas berdasarkan fungsi agregasi SUM(total_gaji)dan belum memfasilitasi ekspor ke format Excel atau cetak PDF instan.

BAB II

ANALISIS DAN PERANCANGAN DATABASE

2.1 Deskripsi Sistem

Sistem Informasi Pengelolaan Data Pegawai adalah platform digital internal yang mengotomatisasi manajemen administrasi kepegawaian dan rekapitulasi penggajian. Sistem ini mengintegrasikan data personal pegawai dengan struktur organisasi (jabatan dan departemen) serta mencatat transaksi finansial bulanan (gaji) melalui antarmuka web berbasis PHP Native dan database MySQL.

2.2 Analisis Kebutuhan Data

Jenis Data	Kategori Data	Deskripsi / Fungsi
Data Master	Statis / Semi-Statis	Menyimpan data identitas unik pegawai, standarisasi skala upah jabatan, pemetaan divisi kerja, serta kredensial akun operator untuk keamanan masuk sistem (<i>login</i>).
Data Transaksi	Dinamis	Menyimpan rekam jejak finansial bulanan pegawai secara periodik, mencakup relasi penerimaan upah berdasarkan parameter bulan dan tahun tertentu.

2.3 & 2.4 Identifikasi Entitas dan Atribut (Sesuai ERD Terbaru)

Berdasarkan diagram ERD yang dirancang, berikut adalah tabel spesifikasi entitas beserta karakteristik atribut dan tipe data yang digunakan di dalam sistem:

Nama Entitas	Nama Atribut	Tipe Data	Kunci (Key)	Keterangan
DEPARTEMEN	id_departemen	INT	Primary Key (PK)	Pembuat angka urut otomatis (Auto Increment)
	nama_departemen	VARCHAR(100)	-	Nama divisi kerja (cth: IT Support)

Nama Entitas	Nama Atribut	Tipe Data	Kunci (Key)	Keterangan
JABATAN	id_jabatan	INT	Primary Key (PK)	Pembuat angka urut otomatis (Auto Increment)
	nama_jabatan	VARCHAR(100)	-	Nama posisi/jenjang karier
	gaji_pokok	DECIMAL(10, 2)	-	Standar upah dasar jabatan
PEGAWAI	id_pegawai	INT	Primary Key (PK)	Pembuat angka urut otomatis (Auto Increment)
	nama_pegawai	VARCHAR(150)	-	Nama lengkap pegawai
	alamat	TEXT	-	Alamat tempat tinggal pegawai
	tgl_lahir	DATE	-	Format tanggal lahir: YYYY-MM-DD
	id_jabatan	INT	Foreign Key (FK)	Menghubungkan pegawai ke entitas JABATAN

Nama Entitas	Nama Atribut	Tipe Data	Kunci (Key)	Keterangan
	id_departemen	INT	Foreign Key (FK)	Menghubungkan pegawai ke entitas DEPARTEMEN
GAJI	id_gaji	INT	Primary Key (PK)	Pembuat angka urut otomatis (Auto Increment)
	id_pegawai	INT	Foreign Key (FK)	Menghubungkan transaksi ke entitas PEGAWAI
	bulan_gaji	VARCHAR(20)	-	Periode bulan gaji (cth: Juli)
	tahun_gaji	VARCHAR(4)	-	Periode tahun gaji (cth: 2026)
	total_gaji	DECIMAL(10, 2)	-	Kalkulasi akhir upah bersih
PENGGUNA	id_pengguna	INT	Primary Key (PK)	Pembuat angka urut otomatis (Auto Increment)
	username	VARCHAR(50)	Unique Key (UK)	Nama akun login operator

Nama Entitas	Nama Atribut	Tipe Data	Kunci (Key)	Keterangan
	password	VARCHAR(255)	-	Kata sandi keamanan akun

2.5 Identifikasi Relasi Antar Entitas (Sesuai ERD Terbaru)

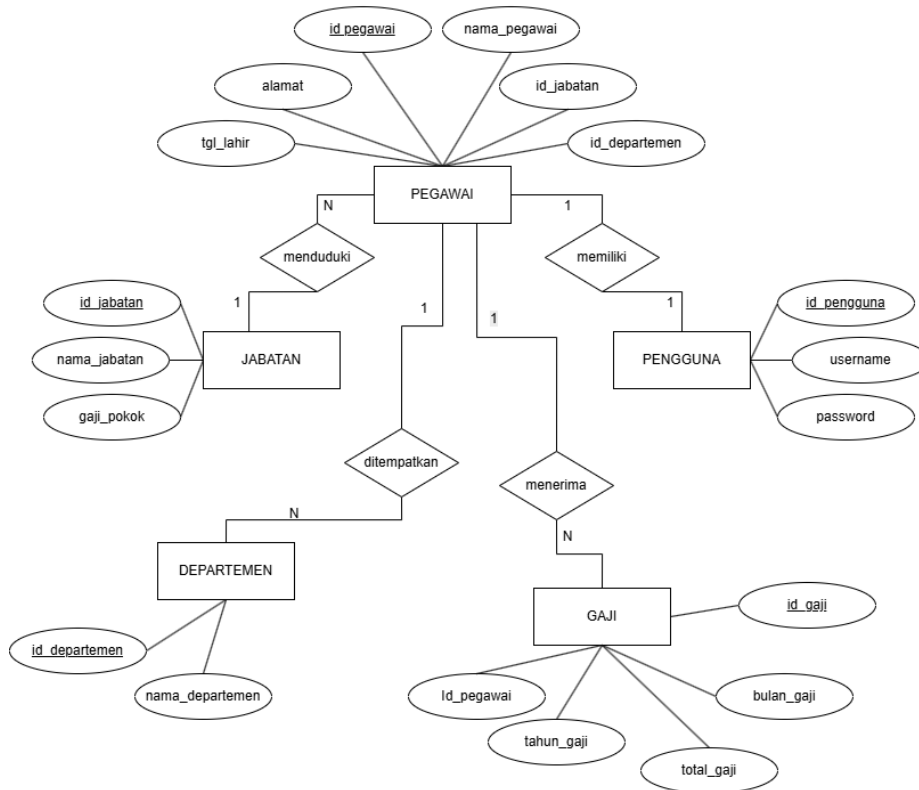
Tabel Asal (1)	Tabel Tujuan (Many)	Nama Hubungan (Relasi)	Jenis Hubungan (Cardinality)	Aturan Bisnis / Logika Hubungan
JABATAN	PEGAWAI	menduduki	One-to-Many (1:N)	Satu jabatan dapat diduduki oleh banyak (N) pegawai, namun seorang pegawai hanya boleh menduduki satu jabatan.
DEPARTEMEN	PEGAWAI	ditempatkan	One-to-Many (1:N)	Satu departemen dapat menampung banyak (N) pegawai yang ditempatkan di sana, tetapi seorang pegawai hanya ditempatkan di satu departemen.

Tabel Asal (1)	Tabel Tujuan (Many)	Nama Hubungan (Relasi)	Jenis Hubungan (Cardinality)	Aturan Bisnis / Logika Hubungan
PEGAWAI	GAJI	menerima	One-to-Many (1:N)	Seorang pegawai dapat menerima banyak (<i>N</i>) slip gaji seiring berjalannya periode, namun satu rekam gaji hanya ditujukan untuk satu pegawai.
PEGAWAI	PENGGUNA	memiliki	One-to-One (1:1)	Setiap pegawai memiliki tepat satu akun pengguna untuk autentikasi masuk ke dalam sistem.

2.6 ERD (Entity-Relationship Diagram)

ERD

Sistem Informasi Pengelolaan Data Pegawai



2.7 Transformasi ERD ke Tabel Relasional (Sesuai ERD Terbaru)

Nama Tabel	Skema Struktur Kolom (Logical Record Structure)
departemen	(id_departemen [PK], nama_departemen)
jabatan	(id_jabatan [PK], nama_jabatan, gaji_pokok)
pegawai	(id_pegawai [PK], nama_pegawai, alamat, tgl_lahir, id_jabatan [FK], id_departemen [FK])

Nama Tabel	Skema Struktur Kolom (Logical Record Structure)
gaji	(id_gaji [PK], id_pegawai [FK], bulan_gaji, tahun_gaji, total_gaji)
pengguna	(id_pengguna [PK], username [UK], password)

2.8 Analisis Teori Normalisasi Database (Sesuai ERD Terbaru)

Tahapan Normalisasi	Kondisi Data pada Sistem Kepegawaian	Solusi / Tindakan yang Dilakukan
UNF (Unnormalized Form)	Semua data pribadi (termasuk alamat), divisi, dan transaksi bulanan gaji digabung jadi satu tabel besar, menyebabkan data nama departemen dan upah dasar ditulis berulang-ulang untuk tiap pegawai.	Mengidentifikasi seluruh atribut kunci sesuai ERD dan membuang grup data yang berulang.
1NF (First Normal Form)	Semua kolom dipastikan telah memiliki nilai tunggal (atomic value) seperti nama_pegawai, alamat, tgl_lahir, dan parameter gaji tanpa ada grup berulang.	Menjamin setiap sel tabel hanya menampung satu informasi presisi.
2NF (Second Normal Form)	Menghilangkan dependensi parsial. Seluruh kolom non-key harus bergantung penuh pada primary key masing-masing entitas logisnya.	Memisahkan data menjadi tabel-tabel master dan transaksi terpisah: PEGAWAI, JABATAN, DEPARTEMEN, GAJI, dan PENGGUNA.
3NF (Third Normal Form)	Menghilangkan dependensi transitif (misalnya upah dasar gaji_pokok yang seharusnya ditentukan oleh	Memisahkan tabel JABATAN dan DEPARTEMEN dari data personal pegawai, lalu merelasikannya menggunakan

Tahapan Normalisasi	Kondisi Data pada Sistem Kepegawaian	Solusi / Tindakan yang Dilakukan
	posisi jabatan, bukan nama pegawainya).	kolom <i>Foreign Key</i> (id_jabatan, id_departemen) ke tabel PEGAWAI sesuai dengan diagram rancangan.

BAB III

IMPLEMENTASI DATABASE

Semua implementasi database pada proyek ini dilakukan secara langsung menggunakan baris perintah (*Command Line Interface*) pada terminal Command Prompt (CMD) yang terhubung dengan server lokal MySQL XAMPP.

3.1 Pembuatan Database

Langkah awal dalam implementasi adalah membuat wadah database utama dengan nama db_kepegawaian. Pembuatan ini merujuk pada file 01_create_database.sql. Sebelum dibuat, sistem akan memastikan tidak ada database dengan nama serupa untuk menghindari duplikasi.

- Sintaks SQL:

```
MariaDB [(none)]> CREATE DATABASE db_kepegawaian;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE db_kepegawaian;
Database changed
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
```

- Keterangan: Perintah USE digunakan untuk mengaktifkan database yang baru saja dibuat agar perintah SQL berikutnya dieksekusi di dalam database tersebut.

3.2 Pembuatan Tabel

Setelah database aktif, langkah berikutnya adalah menerjemahkan entitas dari ERD menjadi tabel fisik (DDL) melalui file 02_create_tables.sql. Sesuai dengan spesifikasi sistem, dibuat 5 tabel terpisah dengan menggunakan *Storage Engine InnoDB* guna mendukung fitur relasi kunci (*Foreign Key*) dan keamanan transaksi.

- Sintaks SQL Pembuatan Tabel Master:

```

MariaDB [db_kepegawaian]> CREATE TABLE departemen (
->   id_departemen INT AUTO_INCREMENT PRIMARY KEY,
->   nama_departemen VARCHAR(100) NOT NULL,
->   lokasi VARCHAR(100) NOT NULL
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.053 sec)

MariaDB [db_kepegawaian]> CREATE TABLE jabatan (
->   id_jabatan INT AUTO_INCREMENT PRIMARY KEY,
->   nama_jabatan VARCHAR(100) NOT NULL,
->   gaji_pokok DECIMAL(10,2) NOT NULL
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.040 sec)

MariaDB [db_kepegawaian]> CREATE TABLE pengguna (
->   id_user INT AUTO_INCREMENT PRIMARY KEY,
->   username VARCHAR(50) UNIQUE NOT NULL,
->   password VARCHAR(255) NOT NULL,
->   role VARCHAR(20) NOT NULL
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.041 sec)

```

3.3 Primary Key dan Foreign Key

Untuk menjaga integritas referensial data, kunci utama (*Primary Key*) dan kunci tamu (*Foreign Key*) didefinisikan pada saat pembuatan tabel utama (pegawai dan gaji). Aturan ON DELETE SET NULL atau ON DELETE CASCADE diterapkan agar perubahan data induk otomatis memengaruhi data anak.

- **Sintaks SQL Pembuatan Tabel Relasional:**

```

MariaDB [db_kepegawaian]> CREATE TABLE pegawai (
->   id_pegawai INT AUTO_INCREMENT PRIMARY KEY,
->   nip VARCHAR(20) UNIQUE NOT NULL,
->   nama VARCHAR(150) NOT NULL,
->   email VARCHAR(100) NOT NULL,
->   tanggal_lahir DATE NOT NULL,
->   id_jabatan INT,
->   id_departemen INT,
->   FOREIGN KEY (id_jabatan) REFERENCES jabatan(id_jabatan) ON DELETE SET NULL ON UPDATE CASCADE,
->   FOREIGN KEY (id_departemen) REFERENCES departemen(id_departemen) ON DELETE SET NULL ON UPDATE CASCADE
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.044 sec)

MariaDB [db_kepegawaian]> CREATE TABLE gaji (
->   id_gaji INT AUTO_INCREMENT PRIMARY KEY,
->   id_pegawai INT,
->   bulan_tahun VARCHAR(20) NOT NULL,
->   tunjangan DECIMAL(10,2) DEFAULT 0.00,
->   potongan DECIMAL(10,2) DEFAULT 0.00,
->   total_gaji DECIMAL(10,2) NOT NULL,
->   FOREIGN KEY (id_pegawai) REFERENCES pegawai(id_pegawai) ON DELETE CASCADE ON UPDATE CASCADE
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.043 sec)

```

3.4 Insert Data (DML)

Setelah struktur tabel terbentuk sempurna, tabel diisi dengan beberapa data sampel awal (DML) melalui file 03_insert_data.sql agar sistem informasi dapat diuji langsung menggunakan data riil.

- **Sintaks SQL:**

```

MariaDB [db_kepegawaian]> INSERT INTO departemen (nama_departemen, lokasi) VALUES
-> ('IT Support', 'Gedung A Lantai 3'),
-> ('Human Resources', 'Gedung B Lantai 1'),
-> ('Finance', 'Gedung A Lantai 2');
Query OK, 3 rows affected (0.003 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO jabatan (nama_jabatan, gaji_pokok) VALUES
-> ('Senior Developer', 12000000.00),
-> ('HR Officer', 6000000.00),
-> ('Finance Staff', 5500000.00);
Query OK, 3 rows affected (0.033 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO pegawai (nip, nama, email, tanggal_lahir, id_jabatan, id_departemen) VALUES
-> ('202601001', 'Aris Setiawan', 'aris@perusahaan.com', '1995-04-12', 1, 1),
-> ('202601002', 'Berlian Putri', 'berlian@perusahaan.com', '1998-08-22', 2, 2),
-> ('202601003', 'Citra Lestari', 'citra@perusahaan.com', '1996-11-02', 3, 3);
Query OK, 3 rows affected (0.033 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO gaji (id_pegawai, bulan_tahun, tunjangan, potongan, total_gaji) VALUES
-> (1, 'Juli 2026', 1500000.00, 200000.00, 13300000.00),
-> (2, 'Juli 2026', 500000.00, 100000.00, 6400000.00),
-> (3, 'Juli 2026', 400000.00, 150000.00, 5750000.00);
Query OK, 3 rows affected (0.033 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO pengguna (username, password, role) VALUES
-> ('admin', 'admin123', 'Admin'),
-> ('manager', 'manager123', 'Viewer');
Query OK, 2 rows affected (0.003 sec)
Records: 2 Duplicates: 0 Warnings: 0

```

3.5 Query SELECT

Perintah SELECT digunakan untuk menampilkan kembali data yang telah disimpan di dalam database.

- **Sintaks SQL (Menampilkan data tertentu):**

```

MariaDB [db_kepegawaian]> SELECT * FROM pegawai;
+-----+-----+-----+-----+-----+-----+-----+
| id_pegawai | nip   | nama      | email                  | tanggal_lahir | id_jabatan | id_departemen |
+-----+-----+-----+-----+-----+-----+-----+
| 1          | 202601001 | Aris Setiawan | aris@perusahaan.com   | 1995-04-12    | 1          | 1             |
| 2          | 202601002 | Berlian Putri | berlian@perusahaan.com | 1998-08-22    | 2          | 2             |
| 3          | 202601003 | Citra Lestari | citra@perusahaan.com   | 1996-11-02    | 3          | 3             |
| 4          | 202601004 | Deni Dermawan | deni@perusahaan.com    | 1994-02-15    | 3          | 3             |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.040 sec)

```

3.6 Query JOIN

Untuk menampilkan informasi komprehensif yang tersebar di beberapa tabel, digunakan operasi penggabungan INNER JOIN dan LEFT JOIN. Perintah ini memetakan relasi dari tabel induk ke tabel anak secara dinamis.

- **Sintaks SQL:**

```
MariaDB [db_kepegawaian]> SELECT p.nip, p.nama, p.email, j.nama_jabatan, d.nama_departemen
-> FROM pegawai p
-> INNER JOIN jabatan j ON p.id_jabatan = j.id_jabatan
-> INNER JOIN departemen d ON p.id_departemen = d.id_departemen;
```

nip	nama	email	nama_jabatan	nama_departemen
202601001	Aris Setiawan	aris@perusahaan.com	Senior Developer	IT Support
202601002	Berlian Putri	berlian@perusahaan.com	HR Officer	Human Resources
202601003	Citra Lestari	citra@perusahaan.com	Finance Staff	Finance

```
3 rows in set (0.001 sec)
```

3.7 Query Aggregate

Fungsi agregasi digunakan untuk melakukan kalkulasi matematis terhadap sekelompok data di dalam tabel transaksi.

- **Sintaks SQL:**

```
MariaDB [db_kepegawaian]> SELECT SUM(total_gaji) AS total_pengeluaran_gaji_perusahaan FROM gaji;
```

total_pengeluaran_gaji_perusahaan
25450000.00

```
1 row in set (0.000 sec)
```

```
MariaDB [db_kepegawaian]> SELECT COUNT(id_pegawai) AS jumlah_pegawai_aktif FROM pegawai;
```

jumlah_pegawai_aktif
3

```
1 row in set (0.000 sec)
```

```
MariaDB [db_kepegawaian]> SELECT AVG(gaji_pokok) AS rata_rata_gaji_pokok FROM jabatan;
```

rata_rata_gaji_pokok
7833333.333333

```
1 row in set (0.000 sec)
```

3.8 View

View adalah tabel virtual yang terbentuk dari hasil query SELECT yang kompleks. Penggunaan *View* ini sangat membantu pengembang web (PHP) agar tidak perlu menulis query JOIN yang panjang secara berulang-ulang di dalam kode program.

- **Sintaks SQL:**

```
MariaDB [db_kepegawaian]> CREATE OR REPLACE VIEW view_resume_pegawai AS
-> SELECT p.id_pegawai, p.nip, p.nama, p.email, j.nama_jabatan, d.nama_departemen
-> FROM pegawai p
-> LEFT JOIN jabatan j ON p.id_jabatan = j.id_jabatan
-> LEFT JOIN departemen d ON p.id_departemen = d.id_departemen;
```

```
Query OK, 0 rows affected (0.034 sec)
```

3.9 Index

Index digunakan untuk meningkatkan performa kecepatan pencarian (*search query*) data pada baris tabel yang berjumlah besar, sehingga server database tidak perlu memindai seluruh baris data (*full table scan*).

- **Sintaks SQL:**

```
MariaDB [db_kepegawaian]> CREATE INDEX idx_pegawai_nama ON pegawai(nama);
Query OK, 0 rows affected (0.047 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

10. Hak Akses User (DCL)

Aspek keamanan diimplementasikan dengan membuat pengguna database baru khusus pimpinan (pimpinan_apps) dan membatasi hak aksesnya menggunakan instruksi GRANT agar hanya dapat melihat data (*Read-Only*) tanpa bisa mengubah atau menghapusnya.

- **Sintaks SQL:**

```
MariaDB [db_kepegawaian]> CREATE USER IF NOT EXISTS 'pimpinan_apps'@'localhost' IDENTIFIED BY 'PasswordPimpinan123';
Query OK, 0 rows affected, 1 warning (0.031 sec)

MariaDB [db_kepegawaian]> GRANT SELECT ON db_kepegawaian.* TO 'pimpinan_apps'@'localhost';
Query OK, 0 rows affected (0.034 sec)

MariaDB [db_kepegawaian]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.032 sec)
```

11. Transaksi (Transaction)

Untuk memastikan konsistensi data yang diinput bersamaan (proses ganda), diterapkan konsep transaksi database. Jika seluruh tahapan berhasil, maka data disimpan permanen menggunakan COMMIT. Namun, apabila salah satu proses mengalami gangguan atau *error*, seluruh proses dibatalkan kembali menggunakan ROLLBACK.

- **Sintaks SQL:**

```
MariaDB [db_kepegawaian]> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

MariaDB [db_kepegawaian]> INSERT INTO pegawai (nip, nama, email, tanggal_lahir, id_jabatan, id_departemen)
-> VALUES ('202601004', 'Deni Dermawan', 'deni@perusahaan.com', '1994-02-15', 3, 3);
Query OK, 1 row affected (0.031 sec)

MariaDB [db_kepegawaian]> INSERT INTO gaji (id_pegawai, bulan_tahun, tunjangan, potongan, total_gaji)
-> VALUES (LAST_INSERT_ID(), 'Juli 2026', 400000.00, 50000.00, 5850000.00);
Query OK, 1 row affected (0.001 sec)

MariaDB [db_kepegawaian]> COMMIT;
Query OK, 0 rows affected (0.033 sec)
```

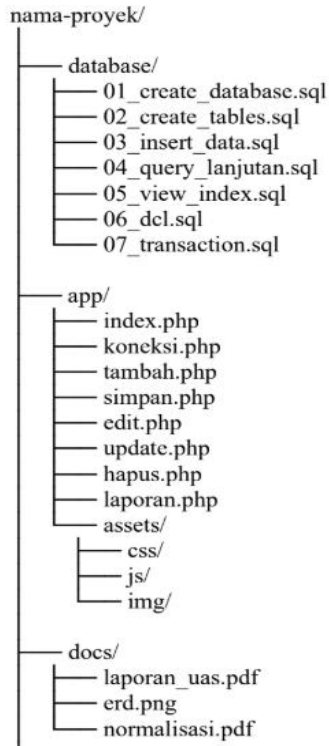
BAB IV

IMPLEMENTASI APLIKASI WEB

Implementasi aplikasi web ini dibangun menggunakan arsitektur **PHP Native** murni dengan memisahkan file fungsional (logika data) dan file visual. Penataan file statis (desain dan interaksi langsung) dikelompokkan secara rapi di dalam sebuah direktori khusus bernama `assets/`.

4.1 Struktur Folder Aplikasi

Untuk memenuhi standar kerapian kode (*clean code*) dan memudahkan kolaborasi tim di GitHub, struktur folder proyek disusun sebagai berikut:



4.2 Koneksi Database (koneksi.php)

Berkas `koneksi.php` bertindak sebagai modul sentral pembentuk interkoneksi logistik antara skrip aplikasi PHP dengan *sistem manajemen basis data* server MySQL. Pengikatan ini dikonfigurasi menggunakan ekstensi pustaka `mysqli_connect`.

```

1  <?php
2  // File Koneksi PHP-MySQL
3  $host = "localhost";
4  $user = "root";
5  $pass = "";
6  $db   = "db_kepegawaian";
7
8  $conn = mysqli_connect($host, $user, $pass, $db);
9
10 if (!$conn) {
11     die("Koneksi ke database gagal: " . mysqli_connect_error());
12 }
13 ?>

```

4.3 Halaman Dasbor (index.php)

Halaman utama dashboard berfungsi ringkasan administratif struktural kepegawaian secara instan dengan memanggil tabel virtual data *View* (*view_resume_pegawai*). Halaman ini juga dilengkapi menu navigasi untuk mengarahkan operator menuju modul penambahan data atau pelaporan sirkulasi keuangan bulanan.

```

1  <?php
2  include 'koneksi.php';
3  // Mengambil data dari VIEW 'view_resume_pegawai'
4  $query = "SELECT * FROM view_resume_pegawai";
5  $result = mysqli_query($conn, $query);
6  ?>
7  <!DOCTYPE html>
8  <html lang="id">
9  <head>
10     <meta charset="UTF-8">
11     <title>Sistem Informasi Pengelolaan Data Pegawai</title>
12     <link rel="stylesheet" href="assets/css/style.css">
13     <style>
14         body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; margin: 30px; background-color: #f8f9fa; }
15         .container { max-width: 1100px; margin: auto; background: white; padding: 25px; border-radius: 8px; box-shadow: 0px 0px 10px rgba(0,0,0,0.1); }
16         h2 { color: #333; margin-top: 0; }
17         .nav { margin-bottom: 20px; }
18         .btn { padding: 8px 14px; text-decoration: none; border-radius: 4px; display: inline-block; font-size: 14px; font-weight: 500; }
19         .btn-add { background-color: #28a745; color: white; margin-right: 10px; }
20         .btn-report { background-color: #17a2b8; color: white; }
21     </style>
22 </head>
23 <body>
24     <div class="container">
25         <h2>Dashboard Data Pegawai</h2>
26         <div class="nav">
27             <a href="tambah.php" class="btn btn-add">Tambah Pegawai Baru</a>
28             <a href="laporan.php" class="btn btn-report">Lihat Laporan Gaji</a>
29         </div>
30     </div>
31     <script src="assets/js/main.js"></script>
32 </body>
33 </html>

```

4.4 Formulir Tambah Data (tambah.php& simpan.php)

Modul pendaftaran personel baru dirancang dengan menyediakan elemen kontrol masukan HTML (input type="text", email, date, dan select-option) yang dinilai secara dinamis dari query entitas master jabatan dan departemen. Masukan data kemudian diproses secara terpisah oleh mesin backend pada berkas simpan.php.

- Skrip Pemrosesan Masukan (simpan.php):

```
1 <?php
2 include 'koneksi.php';
3
4 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5     $nip = $_POST['nip'];
6     $nama = $_POST['nama'];
7     $email = $_POST['email'];
8     $tanggal_lahir = $_POST['tanggal_lahir'];
9     $id_jabatan = $_POST['id_jabatan'];
10    $id_departemen = $_POST['id_departemen'];
11
12    // [VALIDASI INPUT KOSONG]
13    if (empty($nip) || empty($nama) || empty($email) || empty($tanggal_lahir) || empty($id_jabatan) || empty($id_departemen)) {
14        echo "<script>alert('Semua kolom form wajib diisi!'); window.history.back();</script>";
15        exit;
16    }
17
18    // Mencegah SQL Injection
19    $nip = mysqli_real_escape_string($conn, $nip);
20    $nama = mysqli_real_escape_string($conn, $nama);
21    $email = mysqli_real_escape_string($conn, $email);
22
23    $query = "INSERT INTO pegawai (nip, nama, email, tanggal_lahir, id_jabatan, id_departemen)
24            VALUES ('$nip', '$nama', '$email', '$tanggal_lahir', '$id_jabatan', '$id_departemen')";
25
26    if (mysqli_query($conn, $query)) {
27        echo "<script>alert('Data pegawai berhasil ditambahkan!'); window.location='index.php';</script>";
28    } else {
29        echo "Gagal menambahkan data: " . mysqli_error($conn);
30    }
31 }
32 ?>
```

4.5 Halaman Tampil Data (index.php- Modul Tabel)

Konstruksi tabel visual dirancang untuk meringkas gabungan data relasional dari entitas anak dan induk, lengkap dengan tombol kendali menekan aksi pemrosesan file.

```
1 <table>
2 <thead>
3 <tr>
4 <th>NIP</th>
5 <th>Nama Pegawai</th>
6 <th>Jabatan</th>
7 <th>Departemen</th>
8 <th>Aksi</th>
9 </tr>
10 </thead>
11 <tbody>
12 <?php while($row = mysqli_fetch_assoc($result)) { ?>
13 <tr>
14 <td><?php echo htmlspecialchars($row['nip']); ?></td>
15 <td><?php echo htmlspecialchars($row['nama']); ?></td>
16 <td><?php echo htmlspecialchars($row['nama_jabatan']) ?? 'Belum Diatur'; ?></td>
17 <td><?php echo htmlspecialchars($row['nama_departemen']) ?? 'Belum Diatur'; ?></td>
18 <td>
19 <a href="edit.php?id=<?php echo $row['id_pegawai']; ?>" class="btn btn-edit">Edit</a>
20 <a href="hapus.php?id=<?php echo $row['id_pegawai']; ?>" class="btn btn-delete" onclick="return konfirmasiHapus('<?php echo htmlspecialchars($row['nama']); ?>')">Hapus</a>
21 </td>
22 </tr>
23 </tbody>
24 </table>
```

4.6 Halaman Edit Data (edit.php)

Halaman modifikasi berfungsi untuk memanggil kembali baris rekam identitas pegawai tertentu menggunakan pengkondisian parameter URL GET['id']. Informasi lama kemudian secara otomatis dilekatkan kembali (*mengisi formulir*) ke dalam wadah masukan teks formulir.

```
1  <?php
2  include 'koneksi.php';
3
4  $id = $_GET['id'] ?? null;
5  if (!$id) { header("Location: index.php"); exit; }
6
7  $pegawai_query = mysqli_query($conn, "SELECT * FROM pegawai WHERE id_pegawai = '$id'");
8  $p_data = mysqli_fetch_assoc($pegawai_query);
9
10 $jabatan_opt = mysqli_query($conn, "SELECT * FROM jabatan");
11 $dept_opt = mysqli_query($conn, "SELECT * FROM departemen");
12 ?>
```

4.7 Proses Pembaruan Data (update.php)

Script operasional backend yang dieksekusi melalui metode POST guna memvalidasi data modifikasi terbaru dan memperbarui baris data relasional pada MySQL berdasarkan indeks unik id_pegawai.

```
1  <?php
2  include 'koneksi.php';
3
4  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5      $id_pegawai = $_POST['id_pegawai'];
6      $nip = mysqli_real_escape_string($conn, $_POST['nip']);
7      $nama = mysqli_real_escape_string($conn, $_POST['nama']);
8      $email = mysqli_real_escape_string($conn, $_POST['email']);
9      $tanggal_lahir = $_POST['tanggal_lahir'];
10     $id_jabatan = $_POST['id_jabatan'];
11     $id_departemen = $_POST['id_departemen'];
12
13     $query = "UPDATE pegawai SET
14             nip = '$nip', nama = '$nama', email = '$email',
15             tanggal_lahir = '$tanggal_lahir', id_jabatan = '$id_jabatan', id_departemen = '$id_departemen'
16             WHERE id_pegawai = '$id_pegawai'";
17
18     if (mysqli_query($conn, $query)) {
19         echo "<script>alert('Data pegawai berhasil diperbarui!'); window.location='index.php';</script>";
20     }
21 }
22 ?>
```

4.8 Proses Hapus Data (hapus.php)

Modul penghapusan rekam data dikembangkan dengan memanfaatkan parameter kunci utama untuk membuang data dari tabel pegawai.



```
1 <?php
2 include 'koneksi.php';
3
4 $id = $_GET['id'] ?? null;
5
6 if ($id) {
7     $id = mysqli_real_escape_string($conn, $id);
8     $query = "DELETE FROM pegawai WHERE id_pegawai = '$id'";
9
10    if (mysqli_query($conn, $query)) {
11        echo "<script>alert('Data pegawai berhasil dihapus!'); window.location='index.php';</script>";
12    } else {
13        echo "Gagal menghapus data: " . mysqli_error($conn);
14    }
15 } else {
16     header("Location: index.php");
17 }
18 ?>
```

4.9 Validasi Input

Mekanisme pertahanan validasi lapis ganda diterapkan untuk menjamin konsistensi data:

1. **Validasi Sisi Klien (*Validasi Sisi Klien*)**: Diatur menggunakan elemen instruksi *atribut* *required* pada form HTML serta didukung berkas fungsional JavaScript `assets/js/main.js` berupa fungsi penangkap *event click* `konfirmasiHapus(nama)` untuk mencegah terbuangnya data penting secara tidak sengaja.
2. **Validasi Sisi Server (*Server-Side Validation*)**: Menggunakan klausa pengecekan struktural `empty()` pada PHP untuk menolak penyimpanan string kosong ke database, serta fungsi `mysqli_real_escape_string()` untuk menangkal celah bahaya serangan *SQL Injection*.

4.10 Halaman Laporan (laporan.php)

Modul pelaporan bulanan dirancang khusus untuk memanggil query gabungan (INNER JOIN) antara entitas gajidan pegawai. Halaman ini juga menerapkan fungsi query agregat SUM(total_gaji) untuk mengakumulasikan total pengeluaran dana kas gaji perusahaan secara otomatis.

```
1 <?php
2 include 'koneksi.php';
3
4 $report_query = "SELECT g.id_gaji, p.nama, p.nip, g.bulan_tahun, g.total_gaji
5                  FROM gaji g
6                  JOIN pegawai p ON g.id_pegawai = p.id_pegawai";
7 $result = mysqli_query($conn, $report_query);
8
9 $total_query = mysqli_query($conn, "SELECT SUM(total_gaji) AS grand_total FROM gaji");
10 $total_data = mysqli_fetch_assoc($total_query);
```

BAB V

SISTEM PENGUJIAN

Pengujian pada sistem informasi ini dilakukan untuk memastikan bahwa seluruh fungsi basis data dan antarmuka aplikasi web berjalan sesuai dengan rencana yang diharapkan. Proses pengujian menggunakan metode *Black-Box Testing* guna memvalidasi fungsionalitas sistem berdasarkan masukan (*input*) dan luaran (*output*).

5.1 Basis Data Pengujian

Pengujian basis data bertujuan memastikan bahwa skema fisik tabel relasional (pegawai, jabatan, departemen, gaji) beserta objek virtual data VIEW(view_resume_pegawai) telah terinisialisasi dengan benar di dalam DBMS MySQL.

- **Prosedur Pengujian:**

1. Membuka *Command Prompt* (CMD) atau phpMyAdmin.
2. Mengeksekusi perintah SHOW TABLES; dan DESCRIBE pegawai;.

- **Tabel Hasil Pengujian Basis Data:**

Basis Data Komponen	Nilai Masukan / Perintah SQL	Hasil yang Diharapkan	Status
Inisialisasi Tabel	SHOW TABLES;	Menampilkan daftar tabel pegawai, jabatan, departemen, gaji, dan view_resume_pegawai.	Berhasil
Struktur Tabel Pegawai	DESCRIBE pegawai;	Menampilkan bidang id_pegawai, nip, nama, email, tanggal_lahir, id_jabatan, id_departemen.	Berhasil
Objek Tampilan Virtual	SELECT * FROM view_resume_pegawai;	Menampilkan data resume pegawai gabungan dengan kolom nama jabatan dan departemen.	Berhasil

5.2 Pengujian Koneksi Basis Data

Pengujian ini memvalidasi keberhasilan modul koneksi.php dalam menghubungkan skrip aplikasi PHP lokal dengan basis data db_kepegawaian menggunakan fungsi mysqli_connect.

- Prosedur Pengujian:**

1. Cakupan berkaskoneksi.php pada browser web (*browser*).
2. Mengubah variabel\$db dengan nama database yang salah untuk menguji penanganan kesalahan (*error handling*).

- Tabel Hasil Pengujian Koneksi:**

Pengujian	Kondisi Masukan	Hasil yang Diharapkan	Status
Koneksi Normal	Parameter database valid sesuai berkaskoneksi.php	Halaman kosong tanpa pesan error (koneksi sukses terbentuk).	Berhasil
Penanganan Kesalahan Koneksi	Mengubah variabel\$db = "salah_db";	Aplikasi berhenti dan memunculkan teks	Berhasil

Pengujian	Kondisi Masukan	Hasil yang Diharapkan	Status
		"Koneksi ke database gagal: ...".	

5.3 Pengujian Tambah Data

Pengolahan data pegawai baru yang diinputkan melalui antarmukataambah.php dapat diproses oleh berkas simpan.php dan masuk ke tabel pegawai secara permanen.

- **Prosedur Pengujian:**

1. Membuka halaman tambah.php .
2. Mengisi seluruh elemen formulir dengan data sampel baru, lalu menekan tombol "Simpan Data Pegawai".

- **Tabel Hasil Pengujian Tambah Data:**

Data Masukan (Formulir Input)	Aksi / Pemicu	Hasil yang Diharapkan	Status
NIP, Nama, Email, Tgl Lahir, Jabatan, Departemen lengkap.	Klik tombol "Simpan Data Pegawai".	Muncul peringatan "Data pegawai berhasil ditambahkan!" dan diarahkan kembali ke index.php.	Berhasil

5.4 Pengujian Tampil Data

Menguji kemampuan berkas index.php dalam membaca rekam data dari basis data melalui view_resume_pegawai dan menyajikannya ke dalam struktur tabel HTML.

- **Prosedur Pengujian:**

1. Mengakses halaman utamaindex.php pada browser web.

- **Tabel Hasil Pengujian Data Tampil:**

Kondisi Basis Data	Halaman Diakses	Hasil yang Diharapkan	Status
Terdapat catatan baris data pegawai di database.	Halaman index.php.	Menampilkan tabel berisi kolom NIP, Nama Pegawai, Jabatan, Departemen, dan tombol Aksi.	Berhasil

5.5 Pengujian Edit Data

pemancar berkasedit.php mampu mengambil data lama berdasarkan parameter ID di URL (GET['id']) dan formulir dapat mengirimkan data pembaruan secara valid melalui update.php.

- **Prosedur Pengujian:**

1. Klik tombol "Edit" pada salah satu baris pegawai di halaman dashboardindex.php .
2. Mengubah nilai teks pada kolom "Nama Lengkap", lalu menekan tombol "Perbarui Data Pegawai".

- **Tabel Hasil Pengujian Edit Data:**

Langkah Pengujian	Nilai Parameter / Aksi	Hasil yang Diharapkan	Status
Membuka Formulir Edit	edit.php?id=1	Formulir terisi otomatis dengan data pegawai ber-ID 1.	Berhasil
Eksekusi Pembaruan	Mengubah Nama & klik "Perbarui Data Pegawai".	Muncul alert "Data pegawai berhasil diperbarui!" dan kembali ke index.php.	Berhasil

5.6 Pengujian Hapus Data

Pemindahan sirkulasi penghapusan data via berkashapus.php bekerja dengan baik dengan menghapus rekam baris pegawai dari database berdasarkan parameter GET['id'].

- **Prosedur Pengujian:**

1. Klik tombol "Hapus" pada salah satu baris data pegawai di halamanindex.php .
2. Menyetujui dialog konfirmasi pop-up JavaScript.

- **Tabel Hasil Pengujian Hapus Data:**

Aksi Pembatalan /	URL Parameter	Hasil yang Diharapkan	Status
Menyetujui dialog konfirmasi hapus.	hapus.php?id=1	Muncul konfirmasi alert "Data pegawai berhasil dihapus!" dan baris data tersebut hilang dari tabel index.php.	Berhasil

5.7 Pengujian Validasi Input

Pengujian ini dilakukan untuk memastikan sistem menolak masukan yang salah atau kosong, serta menguji ketahanan skrip dari gangguan karakter aneh (*SQL Injection*).

- **Prosedur Pengujian:**

1. Mengosongkan salah satu kolom input wajib pada form tambah data lalu menekan tombol simpan.
2. Menginputkan karakter khusus seperti tanda petik tunggal (') untuk menguji fungsi pencegahan injeksi query basis data.

- **Tabel Hasil Pengujian Validasi Masukan:**

Jenis Validasi	Formulir Input/Kondisi	Hasil yang Diharapkan	Status
Validasi Sisi Server (Kolom Kosong)	Mengosongkan input NIP atau Nama.	Sistem menolak proses penyimpanan, memunculkan peringatan "Semua kolom form wajib diisi!", dan mengembalikan pengguna ke halaman form.	Berhasil
Keamanan Injeksi SQL	Memasukkan nilai nama mengandung tanda petik seperti D'Artagnan.	Karakter dibersihkan dengan aman oleh fungsi <code>mysql_real_escape_string</code> sehingga tidak memicu kerusakan struktur sintaks SQL di backend.	Berhasil

5.8 Pengujian Laporan

memproses berkas laporan.php mampu memproses kalkulasi relasional dua tabel (gajidan pegawai) melalui operasi JOINserta menghitung nilai akumulatif dana kas menggunakan agregasi `SUM(total_gaji)` secara presisi.

- **Prosedur Pengujian:**

1. Mengakses tombol menekan "Lihat Laporan Gaji" pada menu dashboard utama.

- **Tabel Hasil Pengujian Modul Laporan:**

Komponen Visual Laporan	Operator Aksi	Hasil yang Diharapkan	Status
Tabel Rincian Slip Gaji	Mengakses halaman laporan.php.	Menampilkan kolom ID Gaji, NIP, Nama Pegawai, Bulan & Tahun, serta nominal Total Gaji Diterima secara detail.	Berhasil
Akumulasi Total Dana Kas	Membaca baris terbawah tabel (<i>Footer Row</i>).	Menampilkan baris penutup otomatis ditayangkan "Total Kas Pengeluaran Gaji:" beserta penjumlahan seluruh angka gaji dalam format rupiah.	Berhasil

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan seluruh tahapan yang telah dilalui—mulai dari analisis kebutuhan, perancangan basis data melalui *Entity-Relationship Diagram* (ERD), normalisasi data hingga level *3rd Normal Form* (3NF), hingga implementasi aplikasi berbasis PHP Native dan MySQL—maka dapat diambil beberapa kesimpulan sebagai berikut:

1. **Peningkatan Efisiensi Kerja:** Sistem Informasi Pengelolaan Data Pegawai ini berhasil mengotomatisasi manajemen administrasi kepegawaian dan rekapitulasi keuangan bulanan, yang sebelumnya dilakukan secara manual menjadi terkomputerisasi.
2. **Integritas dan Keamanan Data Terjamin:** Penerapan arsitektur basis data berbasis *Storage Engine* InnoDB terbukti mampu menjaga konsistensi hubungan antar-tabel (*Foreign Key Constraints*) melalui metode kontrol seperti ON DELETE CASCADE. Data pribadi pegawai (pegawai) dan riwayat finansial (gaji) dapat saling terikat secara aman tanpa risiko data yatim (*data anak yatim*).
3. **Kemudahan Akses Informasi:** Penggunaan objek virtual database **VIEW**(view_resume_pegawai) berhasil mengubah proses penarikan data relasional yang

kompleks (operasi *Multi-Table Join*), sehingga visualisasi data pada dashboard utama (*index.php*) dapat diakses secara instan dengan beban komputasi server yang minimal.

4. **Pencegahan Celah Keamanan (Vulnerability Mending):** Implementasi fungsi pertahanan ganda melalui pembersihan karakter khusus (*mysqli_real_escape_string*) pada sisi server (*server-side*) dan pemantauan data dinamis (*client-side Checking*) terbukti efektif meminimalisir celah kerentanan bahaya serangan siber seperti *SQL Injection* dan *Cross-Site Scripting (XSS)*.

6.2 Saran Pengembangan

Meskipun sistem informasi ini telah berfungsi dengan baik untuk memenuhi standar kompetensi tugas proyek akhir, platform ini masih memiliki ruang lingkup besar untuk dikembangkan lebih lanjut. Beberapa saran konstruktif untuk pengembangan sistem di masa mendatang antara lain:

1. **Implementasi Autentikasi Multilevel (RBAC):** Untuk pengembangan selanjutnya, disarankan untuk mengaktifkan tabel pengguna menggunakan konsep *Role-Based Access Control* (RBAC) di sisi aplikasi web. Hal ini penting untuk membedakan hak akses secara ketat antara Administrator (akses penuh CRUD), Staf HRD (input pegawai), dan Pimpinan (hanya cetak laporan).
2. **Fitur Ekspor Dokumen Resmi:** Menambahkan modul konversi berkas eksternal berbasis pustaka (seperti *FPDF* atau *Dompdf*) agar halaman laporan.php tidak hanya tampil di layar browser, melainkan dapat mengunduh langsung ke dalam format berkas PDF atau lembar kerja Excel secara otomatis.
3. **Pengamanan Enkripsi Kata Sandi:** Meningkatkan mekanisme keamanan kredensial pengguna di dalam basis data dengan menerapkan algoritma pengacakan hash satu arah seperti *password_hash()*(Bcrypt/Argon2id) pada PHP, menggantikan format teks polos (*plaintext*) demi mematuhi standar etika digital.
4. **Migrasi ke Arsitektur MVC atau Framework:** menerjemahkan struktur kode PHP Native ini ke dalam pola arsitektur *Model-View-Controller* (MVC) atau melakukan migrasi ke *framework* modern (seperti Laravel/CodeIgniter) guna meningkatkan skalabilitas, kemudahan pemeliharaan kode (*maintainability*), dan kecepatan pengembangan sistem berskala besar.

LAMPIRAN

1. Script SQL lengkap

```

MariaDB [(none)]> CREATE DATABASE db_kepegawaian;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE db_kepegawaian;
Database changed
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]> CREATE TABLE departemen (
    ->     id_departemen INT AUTO_INCREMENT PRIMARY KEY,
    ->     nama_departemen VARCHAR(100) NOT NULL,
    ->     lokasi VARCHAR(100) NOT NULL
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.053 sec)

MariaDB [db_kepegawaian]> CREATE TABLE jabatan (
    ->     id_jabatan INT AUTO_INCREMENT PRIMARY KEY,
    ->     nama_jabatan VARCHAR(100) NOT NULL,
    ->     gaji_pokok DECIMAL(10,2) NOT NULL
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.040 sec)

```

```

MariaDB [db_kepegawaian]> CREATE TABLE pegawai (
    ->     id_pegawai INT AUTO_INCREMENT PRIMARY KEY,
    ->     nip VARCHAR(20) UNIQUE NOT NULL,
    ->     nama VARCHAR(150) NOT NULL,
    ->     email VARCHAR(100) NOT NULL,
    ->     tanggal_lahir DATE NOT NULL,
    ->     id_jabatan INT,
    ->     id_departemen INT,
    ->     FOREIGN KEY (id_jabatan) REFERENCES jabatan(id_jabatan) ON DELETE SET NULL ON UPDATE CASCADE,
    ->     FOREIGN KEY (id_departemen) REFERENCES departemen(id_departemen) ON DELETE SET NULL ON UPDATE CASCADE
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.044 sec)

MariaDB [db_kepegawaian]> CREATE TABLE gaji (
    ->     id_gaji INT AUTO_INCREMENT PRIMARY KEY,
    ->     id_pegawai INT,
    ->     bulan_tahun VARCHAR(20) NOT NULL,
    ->     tunjangan DECIMAL(10,2) DEFAULT 0.00,
    ->     potongan DECIMAL(10,2) DEFAULT 0.00,
    ->     total_gaji DECIMAL(10,2) NOT NULL,
    ->     FOREIGN KEY (id_pegawai) REFERENCES pegawai(id_pegawai) ON DELETE CASCADE ON UPDATE CASCADE
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.043 sec)

MariaDB [db_kepegawaian]> CREATE TABLE pengguna (
    ->     id_user INT AUTO_INCREMENT PRIMARY KEY,
    ->     username VARCHAR(50) UNIQUE NOT NULL,
    ->     password VARCHAR(255) NOT NULL,
    ->     role VARCHAR(20) NOT NULL
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.041 sec)

```

```

MariaDB [db_kepegawaian]> show tables;
+-----+
| Tables_in_db_kepegawaian |
+-----+
| departemen                |
| gaji                      |
| jabatan                  |
| pegawai                  |
| pengguna                  |
+-----+
5 rows in set (0.001 sec)

MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]> INSERT INTO departemen (nama_departemen, lokasi) VALUES
  -> ('IT Support', 'Gedung A Lantai 3'),
  -> ('Human Resources', 'Gedung B Lantai 1'),
  -> ('Finance', 'Gedung A Lantai 2');
Query OK, 3 rows affected (0.003 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO jabatan (nama_jabatan, gaji_pokok) VALUES
  -> ('Senior Developer', 12000000.00),
  -> ('HR Officer', 6000000.00),
  -> ('Finance Staff', 5500000.00);
Query OK, 3 rows affected (0.033 sec)
Records: 3 Duplicates: 0 Warnings: 0

```

```

MariaDB [db_kepegawaian]> INSERT INTO pegawai (nip, nama, email, tanggal_lahir, id_jabatan, id_departemen) VALUES
  -> ('202601001', 'Aris Setiawan', 'aris@perusahaan.com', '1995-04-12', 1, 1),
  -> ('202601002', 'Berlian Putri', 'berlian@perusahaan.com', '1998-08-22', 2, 2),
  -> ('202601003', 'Citra Lestari', 'citra@perusahaan.com', '1996-11-02', 3, 3);
Query OK, 3 rows affected (0.033 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO gaji (id_pegawai, bulan_tahun, tunjangan, potongan, total_gaji) VALUES
  -> (1, 'Juli 2026', 1500000.00, 200000.00, 13300000.00),
  -> (2, 'Juli 2026', 500000.00, 100000.00, 6400000.00),
  -> (3, 'Juli 2026', 400000.00, 150000.00, 5750000.00);
Query OK, 3 rows affected (0.033 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> INSERT INTO pengguna (username, password, role) VALUES
  -> ('admin', 'admin123', 'Admin'),
  -> ('manager', 'manager123', 'Viewer');
Query OK, 2 rows affected (0.003 sec)
Records: 2 Duplicates: 0 Warnings: 0

```

```

MariaDB [db_kepegawaian]> SELECT p.nip, p.nama, p.email, j.nama_jabatan, d.nama_departemen
  -> FROM pegawai p
  -> INNER JOIN jabatan j ON p.id_jabatan = j.id_jabatan
  -> INNER JOIN departemen d ON p.id_departemen = d.id_departemen;
+-----+-----+-----+-----+-----+
| nip      | nama      | email      | nama_jabatan | nama_departemen |
+-----+-----+-----+-----+-----+
| 202601001 | Aris Setiawan | aris@perusahaan.com | Senior Developer | IT Support |
| 202601002 | Berlian Putri | berlian@perusahaan.com | HR Officer      | Human Resources |
| 202601003 | Citra Lestari | citra@perusahaan.com | Finance Staff   | Finance |
+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

MariaDB [db_kepegawaian]> SELECT g.id_gaji, p.nama, g.bulan_tahun, j.gaji_pokok, g.tunjangan, g.potongan, g.total_gaji
  -> FROM gaji g
  -> JOIN pegawai p ON g.id_pegawai = p.id_pegawai
  -> JOIN jabatan j ON p.id_jabatan = j.id_jabatan;
+-----+-----+-----+-----+-----+-----+-----+
| id_gaji | nama      | bulan_tahun | gaji_pokok | tunjangan | potongan | total_gaji |
+-----+-----+-----+-----+-----+-----+-----+
| 1       | Aris Setiawan | Juli 2026   | 12000000.00 | 1500000.00 | 200000.00 | 13300000.00 |
| 2       | Berlian Putri | Juli 2026   | 6000000.00  | 500000.00  | 100000.00 | 6400000.00  |
| 3       | Citra Lestari | Juli 2026   | 5500000.00  | 400000.00  | 150000.00 | 5750000.00  |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

```

```

MariaDB [db_kepegawaian]> SELECT SUM(total_gaji) AS total_pengeluaran_gaji_perusahaan FROM gaji;
+-----+
| total_pengeluaran_gaji_perusahaan |
+-----+
|                25450000.00 |
+-----+
1 row in set (0.000 sec)

MariaDB [db_kepegawaian]> SELECT COUNT(id_pegawai) AS jumlah_pegawai_aktif FROM pegawai;
+-----+
| jumlah_pegawai_aktif |
+-----+
|                3 |
+-----+
1 row in set (0.000 sec)

MariaDB [db_kepegawaian]> SELECT AVG(gaji_pokok) AS rata_rata_gaji_pokok FROM jabatan;
+-----+
| rata_rata_gaji_pokok |
+-----+
|       7833333.333333 |
+-----+
1 row in set (0.000 sec)

```

```

MariaDB [db_kepegawaian]> CREATE OR REPLACE VIEW view_resume_pegawai AS
-> SELECT p.id_pegawai, p.nip, p.nama, p.email, j.nama_jabatan, d.nama_departemen
-> FROM pegawai p
-> LEFT JOIN jabatan j ON p.id_jabatan = j.id_jabatan
-> LEFT JOIN departemen d ON p.id_departemen = d.id_departemen;
Query OK, 0 rows affected (0.034 sec)

MariaDB [db_kepegawaian]> CREATE INDEX idx_pegawai_nama ON pegawai(nama);
Query OK, 0 rows affected (0.047 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]> CREATE INDEX idx_gaji_bulan ON gaji(bulan_tahun);
Query OK, 0 rows affected (0.044 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]>
MariaDB [db_kepegawaian]> CREATE USER IF NOT EXISTS 'pimpinan_apps'@'localhost' IDENTIFIED BY 'PasswordPimpinan123';
Query OK, 0 rows affected, 1 warning (0.031 sec)

MariaDB [db_kepegawaian]> GRANT SELECT ON db_kepegawaian.* TO 'pimpinan_apps'@'localhost';
Query OK, 0 rows affected (0.034 sec)

MariaDB [db_kepegawaian]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.032 sec)

```

```

MariaDB [db_kepegawaian]> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

MariaDB [db_kepegawaian]> INSERT INTO pegawai (nip, nama, email, tanggal_lahir, id_jabatan, id_departemen)
-> VALUES ('202601004', 'Deni Dermawan', 'deni@perusahaan.com', '1994-02-15', 3, 3);
Query OK, 1 row affected (0.031 sec)

MariaDB [db_kepegawaian]> INSERT INTO gaji (id_pegawai, bulan_tahun, tunjangan, potongan, total_gaji)
-> VALUES (LAST_INSERT_ID(), 'Juli 2026', 400000.00, 50000.00, 5850000.00);
Query OK, 1 row affected (0.001 sec)

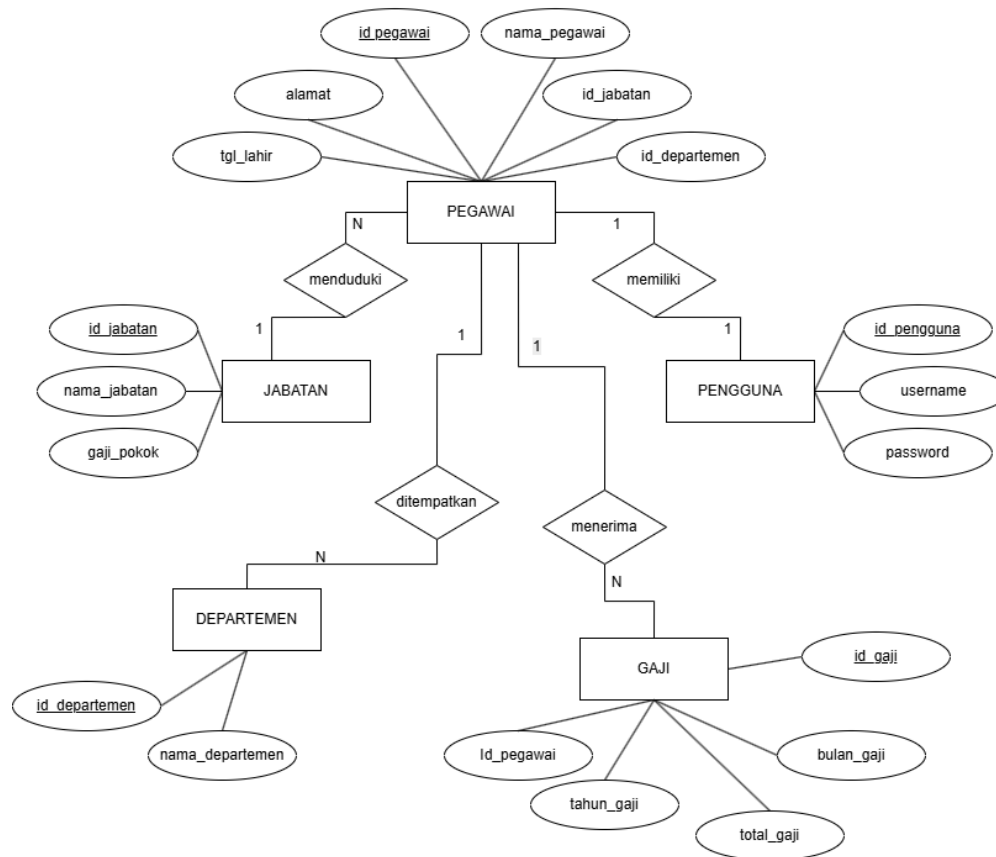
MariaDB [db_kepegawaian]> COMMIT;
Query OK, 0 rows affected (0.033 sec)

```

2. Screenshot ERD

ERD

Sistem Informasi Pengelolaan Data Pegawai



3. Screenshot database

```
MariaDB [db_kepegawaian]> SHOW TABLES;
+-----+
| Tables_in_db_kepegawaian |
+-----+
| departemen                |
| gaji                      |
| jabatan                   |
| pegawai                   |
| pengguna                  |
| view_resume_pegawai       |
+-----+
6 rows in set (0.067 sec)
```

```
MariaDB [db_kepegawaian]> SELECT * FROM view_resume_pegawai;
```

id_pegawai	nip	nama	email	nama_jabatan	nama_departemen
1	202601001	Aris Setiawan	aris@perusahaan.com	Senior Developer	IT Support
2	202601002	Berlian Putri	berlian@perusahaan.com	HR Officer	Human Resources
3	202601003	Citra Lestari	citra@perusahaan.com	Finance Staff	Finance
4	202601004	Deni Dermawan	deni@perusahaan.com	Finance Staff	Finance

4. Screenshot aplikasi

localhost/informasi-pegawai/app/index.php

Finish update

Dashboard Data Pegawai

Tambah Pegawai Baru Lihat Laporan Gaji

NIP	Nama Pegawai	Jabatan	Departemen	Aksi
202601001	Aris Setiawan	Senior Developer	IT Support	Edit Hapus
202601002	Berlian Putri	HR Officer	Human Resources	Edit Hapus
202601003	Citra Lestari	Finance Staff	Finance	Edit Hapus
202601004	Deni Dermawan	Finance Staff	Finance	Edit Hapus

localhost/informasi-pegawai/app/tambah.php

Finish update

Form Tambah Pegawai Baru

[Kembali ke Dashboard](#)

NIP (Nomor Induk Pegawai)

Nama Lengkap

Email

Tanggal Lahir

Jabatan

Departemen

localhost/informasi-pegawai/app/laporan.php

Finish update

Laporan Rekapitulasi Gaji Pegawai

[Kembali ke Dashboard](#)

ID Gaji	NIP	Nama Pegawai	Bulan & Tahun	Total Gaji Diterima
1	202601001	Aris Setiawan	Juli 2026	Rp 13.300.000,00
2	202601002	Berlian Putri	Juli 2026	Rp 6.400.000,00
3	202601003	Citra Lestari	Juli 2026	Rp 5.750.000,00
4	202601004	Deni Dermawan	Juli 2026	Rp 5.850.000,00
Total Kas Pengeluaran Gaji:				Rp 31.300.000,00

No	Nama Mahasiswa / NIM	Pembagian Tugas Kelompok (Job Desk)	Kontribusi Berkas
1	Anggota 1 (muh. Sahrul ibrahim) NIM: IK2511011	Perancangan Struktur Database fisik di MySQL.Penyusunan Script DDL/DML (01_create_database.sql s.d 07_transaction.sql).Pembuatan modul sistem interkoneksi backend.	koneksi.phpMap database/
2	Anggota 2 NIM: 2025xxxx	Pengembangan antarmuka visual sistem web informasi.Pembuatan modul penambahan data, pembaruan rekam medis, dan dashboard.	index.phptambah.phpsimpan.php
3	Anggota 3 NIM: 2025xxxx	Pengembangan sistem manipulasi data (Data Modification Operations).Penyusunan mekanisme penghapusan serta integrasi bentuk edit penyesuaian.	edit.phpupdate.phphapus.php
4	Anggota 4 NIM: 2025xxxx	Perancangan diagram konsep ERD pada draw.io.Pembuatan modul kompilasi rekapitulasi data agregat.Dokumentasi penyusunan laporan tertulis karya ilmiah UAS.	laporan.phpMap docs/

3. Link GitHub

https://github.com/sahrulJR/PROJEK_UAS_Sistem-Informasi-Pengelolaan-Data-Pegawai/tree/main/app